UNIVERSITY OF ENGINEERING & TECHNOLOGY MARDAN

COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

SUBMITTED BY

NAME : RIFAQ AJMAL

REG NO : 23MDBCS 435

SECTION : CS-A

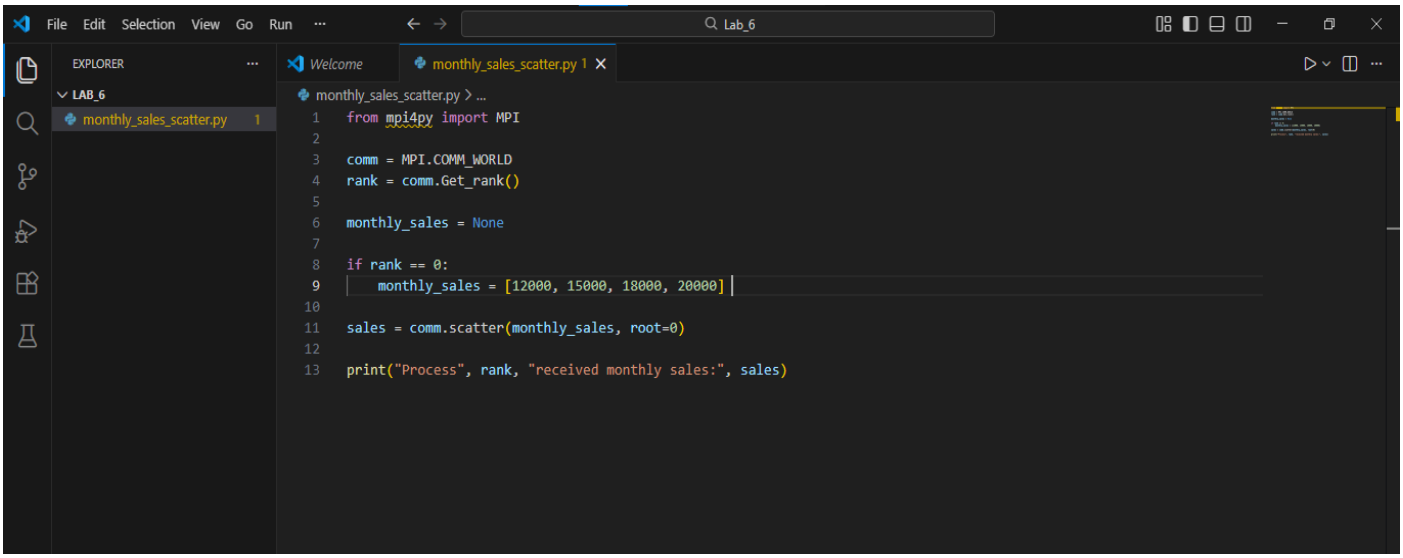
SEMESTER : 6th

Date: 1/3/2026

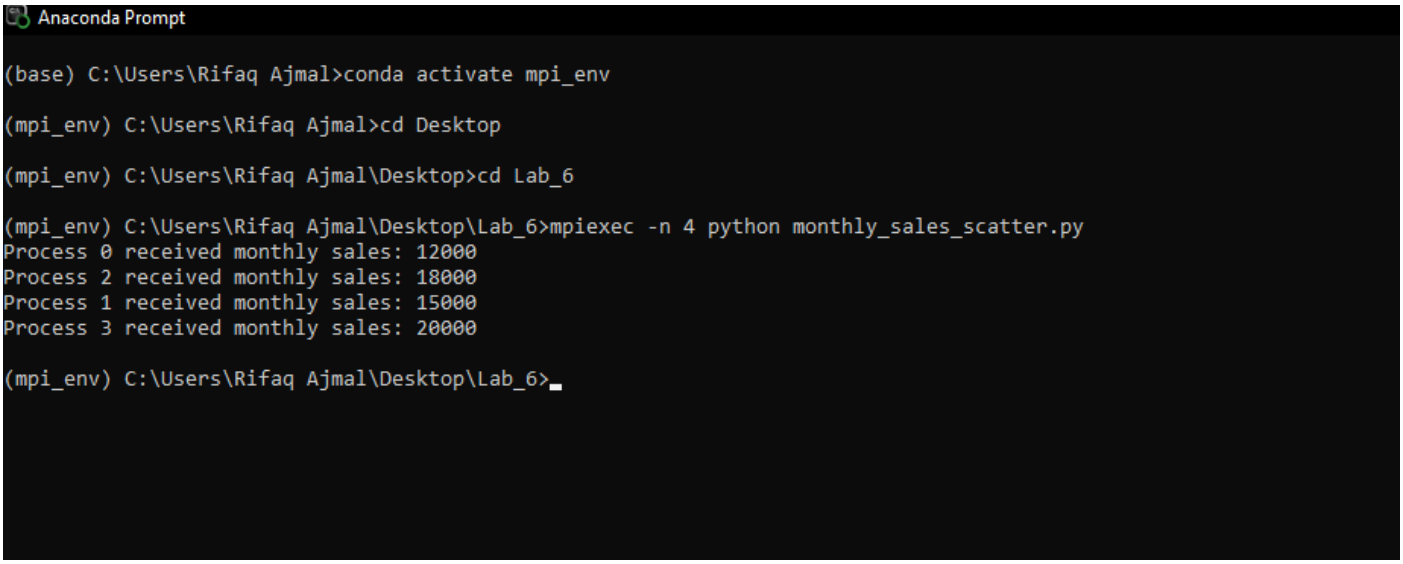
Lab 6 (5a)

Task 1

Modify sales data distribution to monthly reports (MPI_Scatter)

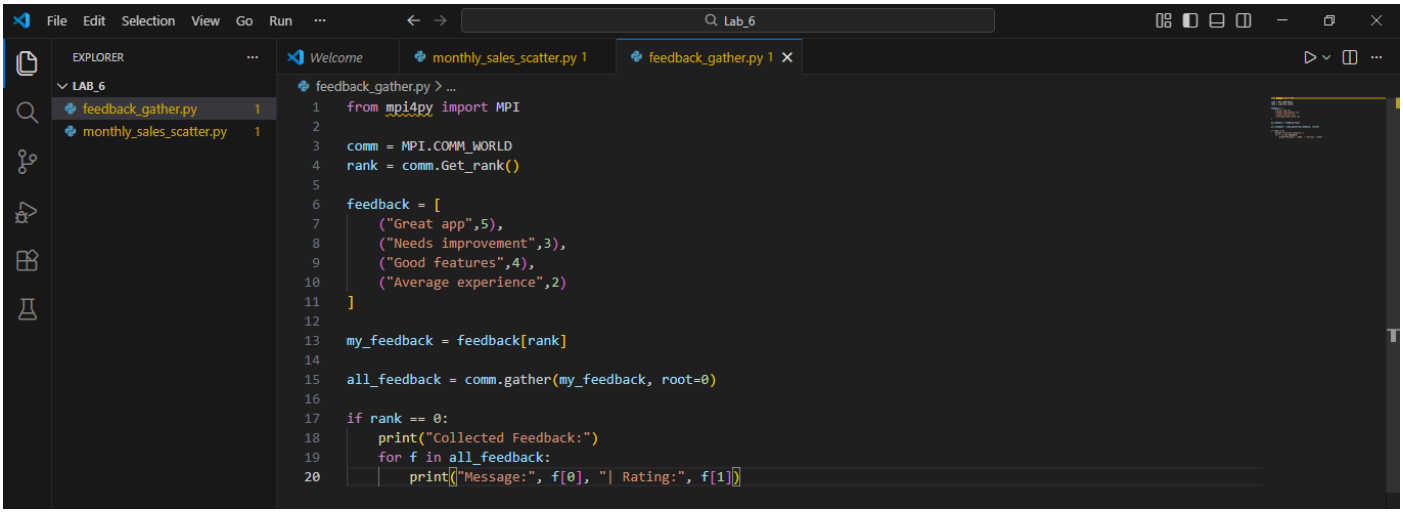


Output



Task 2

User feedback with rating + text (MPI_Gather)



```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

feedback = [
    ("Great app",5),
    ("Needs improvement",3),
    ("Good features",4),
    ("Average experience",2)
]

my_feedback = feedback[rank]

all_feedback = comm.gather(my_feedback, root=0)

if rank == 0:
    print("Collected Feedback:")
    for f in all_feedback:
        print("Message:", f[0], "| Rating:", f[1])
```

Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpiexec -n 4 python feedback_gather.py
Collected Feedback:
Message: Great app | Rating: 5
Message: Needs improvement | Rating: 3
Message: Good features | Rating: 4
Message: Average experience | Rating: 2

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```

Task 3

Custom reduction to find median revenue.

```python
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

revenue = rank * 1000 + 2000

all_revenues = comm.gather(revenue, root=0)

if rank == 0:
    median = np.median(all_revenues)
    print("Revenues:", all_revenues)
    print("Median revenue:", median)
```

Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpiexec -n 4 python median_reduction.py
Revenues: [2000, 3000, 4000, 5000]
Median revenue: 3500.0

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```

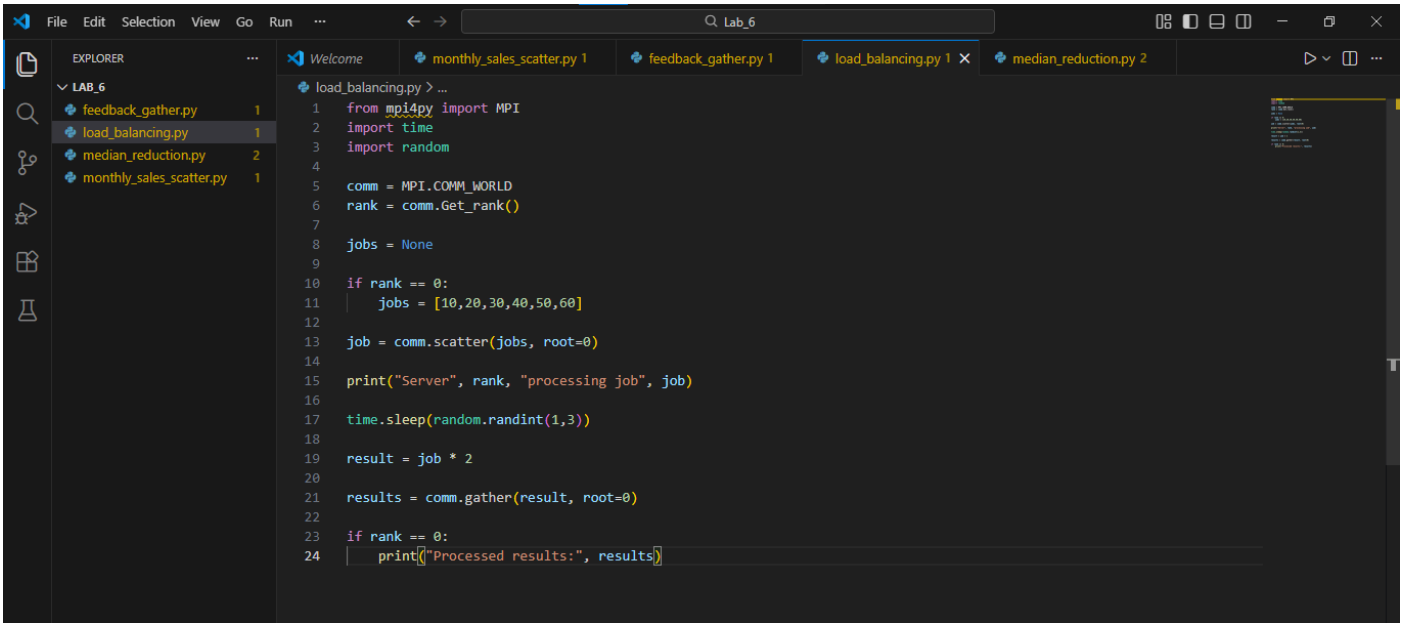
Task 4

If some servers have delayed responses

If some servers respond slowly, synchronization methods such as MPI_Barrier or timeout handling can be used. The system can wait for all servers before calculating results or ignore delayed responses after a certain time limit. Load balancing can also help distribute work more efficiently.

Task 5 (Challenge)

Dynamic load balancing



Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpiexec -n 6 python load_balancing.py
Server 1 processing job 20
Server 3 processing job 40
Server 2 processing job 30
Server 5 processing job 60
Server 4 processing job 50
Server 0 processing job 10
Processed results: [20, 40, 60, 80, 100, 120]

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```